Projet 8 – Guide de travail en équipe

Oct 10, 2026 Gestion du personnel et de la paie

Pour réussir à quatre, on s'accorde d'abord sur des règles communes (base de données, conventions, fonctions partagées), puis chacun développe un domaine complet, et on fusionne chaque semaine.

1. Phase 0 : construire le socle ensemble

Avant de se séparer, les quatre construisent la base commune pendant quelques jours. Sans elle, personne ne peut tester son travail.

- **Dépôt Git** avec la structure MVC vide.
- **Script SQL** des 25 tables et données de démo (un DG, un compte RH, un chef de département, quelques employés, départements, postes, types de congé, jours fériés, rubriques de paie). Une fois validé, on le **gèle** : toute modification passe par une seule personne et est annoncée au groupe.
- **Conventions** écrites dans un `README` : langue des noms (français ou anglais, pas les deux), PDO partout, requêtes préparées, échappement des sorties, noms des statuts et des rôles (`employe`, `chef`, `rh`, `dg`).
- **Squelette technique** : connexion PDO, routeur simple, layout Bootstrap commun avec un menu par rôle, fonction `exiger_role('rh')` pour protéger les pages.
- **Maquettes Figma** : elles servent de référence d'écrans. Chaque responsable s'y rapporte pour son domaine ; un écran qui manque (RH, DG) est dessiné par le responsable du domaine avant d'être codé.

À la fin, chacun lance le projet chez lui et se connecte avec des comptes de démo.

2. Qui fait quoi : un domaine complet par personne

Chacun prend un domaine de bout en bout (base de données, PHP, écran), au lieu de se spécialiser en « front », « back » ou « SQL ». Il y a un **responsable unique par domaine** : sur un sujet, une seule personne tranche. Les 25 tables sont réparties une seule fois : chaque table a un seul propriétaire, les autres la lisent.

Personne	Domaine	Tables	Maquettes rattachées
A	Fondations et sécurité	5	Menus et mobile, Accueil, Profil (mot de passe)
B	Organisation	7	Profil, Documents, Organigramme, Mon équipe
C	Congés et absences	6	Congés, Absences, Avis sur un congé, Calendrier d'équipe
D	Paie	7	Fiches de paie, Avances, Heures supplémentaires

A · Fondations et sécurité

FONCTIONNALITÉS

- Connexion, déconnexion, sessions, jeton CSRF, rôles (`exiger_role`).
- Création et validation des comptes, mot de passe oublié et réinitialisation.
- Notifications, journal des actions, paramètres du système.
- Service d'e-mail (PHPMailer), layout commun, menus par rôle, portail mobile.

TABLES

- `Utilisateur`, `JetonReinitialisation`, `Notification`, `JournalAction`, `ParametreSysteme`

MAQUETTES

- Menus ordinateur et navigation mobile (menu « Plus » à dessiner).
- « Accueil » : structure et cloche de notifications ; chacun y branche ses blocs (solde de congés, dernière fiche, absence à justifier).
- « Profil » : changement de mot de passe.
- À dessiner : connexion, mot de passe oublié, journal et paramètres (RH / DG).

B · Organisation

FONCTIONNALITÉS

- Employés (photo, pièces), départements, postes, contrats.
- Recherche dynamique (fetch) et organigramme.
- Demandes de modification du profil ; demandes de documents et attestation Word.
- Dépôt et téléchargement des documents de l'employé.

TABLES

- `Employe`, `Contrat`, `Departement`, `Poste`, `DemandeModification`, `DocumentEmploye`, `DemandeDocument`

MAQUETTES

- Employé : « Profil », « Documents », « Organigramme ».
- Chef : « Mon équipe » (liste des membres).
- À dessiner : fiches employés, départements, postes et contrats (RH).

C · Congés et absences

FONCTIONNALITÉS

- Demande de congé avec calcul en direct des jours ouvrables, solde, jours fériés.
- Double validation : avis du chef, puis décision de la RH ; e-mail de décision.
- Absences : marquées par le chef (non justifiées), justificatif déposé par l'employé, validation par le chef.
- Calendrier d'équipe.

TABLES

- `DemandeConge`, `ValidationConge`, `SoldeConge`, `TypeConge`, `JourFerie`, `Absence`

MAQUETTES

- Employé : « Congés », « Absences » (à créer).
- Chef : « Avis sur un congé » (avec la liste des demandes à examiner), « Absences », « Justificatif d'absence », « Calendrier d'équipe ».
- « Mon équipe » : boutons d'accès aux demandes et aux absences (à ajouter).
- À dessiner : décision finale sur les congés et types de congé (RH).

D · Paie

FONCTIONNALITÉS

- Rubriques de paie, rubriques par employé et par poste.
- Calcul de la fiche, cycle brouillon → validée par la RH, export PDF, envoi par e-mail.
- Heures supplémentaires : saisies et validées par le chef, contrôlées par la RH, intégrées à la paie.
- Avances sur salaire (validées par la RH) ; retenue pour absence non justifiée, corrigeable par la RH.
- Tableau de bord, une fois le noyau terminé.

TABLES

- `FichePaie`, `LignePaie`, `RubriquePaie`, `EmployeRubrique`, `PosteRubrique`, `HeureSupplementaire`, `AvanceSalaire`

MAQUETTES

- Employé : « Fiches de paie », « Avances ».
- Chef : « Heures supplémentaires ».
- À dessiner : rubriques, génération des fiches, contrôle des heures supplémentaires, avances, tableau de bord (RH / DG).

Les fonctionnalités restantes (tableau de bord, extras) vont à celui qui finit le premier. Les prénoms sont à remplir par l'équipe, selon les envies et les points forts. La charge est équilibrée à peu près à 6 ou 7 tables chacun, A ayant moins de tables mais le chemin critique : il doit finir l'authentification en premier, puis aide B ou D.

3. Faire fonctionner son code avec celui des autres

Trois mécanismes évitent les mauvaises surprises à l'assemblage.

1. **La base de données est le contrat principal.** Si C écrit `demande_conge.statut = 'en_attente_chef'` et que D lit la même table, ils sont d'accord, parce que le schéma est le même pour tous. Même principe pour `absence.justifiee` : C l'écrit, D le lit pour calculer la retenue.
2. **Les fonctions partagées ont une signature convenue dès le premier jour**, même si leur corps est vide au début. Les autres peuvent déjà les appeler.
3. **Les réponses fetch ont un format unique** : `{ "ok": true, "data": ... }` ou `{ "ok": false, "erreur": "..." }`.

Exemples de signatures à convenir :

```
// C écrit, les autres utilisent
SoldeService::joursOuvrables(DateTime $debut, DateTime $fin): float
SoldeService::soldeDisponible(int $employeId, int $typeId, int $annee): float
Absence::nonJustifieesDuMois(int $employeId, int $mois, int $annee): float   // jours, lu
par D

// B écrit, les autres utilisent
Employe::trouver(int $id): ?array
Employe::equipeDuChef(int $chefId): array

// A écrit, tout le monde utilise
Mailer::envoyer(string $dest, string $sujet, string $html, ?string $pj = null): bool
Notification::creer(int $userId, string $titre, string $message, string $lien): void
Journal::ecrire(string $action, string $entite, int $id): void
```

Pour ne pas attendre les autres, chacun développe avec les données de démo : D peut calculer une paie sans que le module employés soit fini.

4. Structure PHP simple et fetch

`fetch` remplace AJAX : même idée (appeler le serveur sans recharger la page), syntaxe plus simple, sans jQuery. Le cahier des charges exige seulement « au moins 3 interactions sans rechargement ». Exemples dans ce projet : recherche d'employés (B), calcul du nombre de jours de la demande de congé (C), aperçu de la fiche de paie (D).

```
projet/
├── config/        db.php (connexion PDO), config.php (local, hors Git)
├── models/        Employe.php, Conge.php, Absence.php, Paie.php (requêtes SQL)
├── controllers/   traitement des requêtes
├── views/         le HTML avec un peu de PHP (un sous-dossier par domaine)
├── api/           scripts appelés par fetch (renvoient du JSON)
├── services/      Mailer, Solde, PDF...
├── public/        css/, js/, images/ (seul dossier accessible)
└── uploads/       fichiers privés (photos, justificatifs), hors de public/
```

Trois règles :

- Le SQL reste dans `models/` : une page ne contient jamais de `SELECT`.

- Chaque page protégée commence par `exiger_role(...)`.
- Chaque script `api/` renvoie du JSON et vérifie lui aussi le rôle et le jeton CSRF : un bouton caché n'empêche personne d'appeler l'URL. Un chef ne peut agir que sur son département, et un employé que sur ses propres données.

Exemple côté PHP (`api/employes_recherche.php`) :

```
<?php
require '../config/init.php';
exiger_role('rh');
$employes = Employe::rechercher($_GET['departement'] ?? null, $_GET['statut'] ?? null);
header('Content-Type: application/json');
echo json_encode(['ok' => true, 'data' => $employes]);
```

Exemple côté JavaScript :

```
async function charger() {
  const params = new URLSearchParams({ departement: select.value });
  const rep = await fetch('/api/employes_recherche.php?' + params);
  const json = await rep.json();
  if (json.ok) afficher(json.data);
}
select.addEventListener('change', charger);
```

Composer et configuration locale : une seule personne (A) installe PHPMailer, Dompdf et PhpWord avec `composer require` et pousse `composer.json` ; les autres lancent `composer install`. Ne poussez pas `vendor/`. Mettez `config.example.php` dans Git et laissez chacun créer son `config.php` local (dans `.gitignore`).

5. Organisation Git

La branche `main` contient toujours une version qui fonctionne.

- Personne ne pousse directement sur `main`.
- Une branche par tâche : `feature/conges-demande`, `feature/absences-justificatif`, `feature/paie-pdf`.
- Fusion par pull request, relue par un autre membre, qui vérifie que ça tourne et que les conventions sont respectées.
- Chacun reste dans son dossier et ses tables autant que possible. Les conflits arrivent surtout sur les fichiers partagés (layout, menu, routeur, `Accueil`) : prévenez le groupe avant de les modifier.
- Petits commits, fusions fréquentes : une branche qui vit trois semaines devient impossible à fusionner.

6. Rythme de travail

- Un tableau de tâches (GitHub Projects ou Trello) avec À faire / En cours / Terminé.
- Chaque tâche a un responsable et une définition de « terminé » (par exemple : la demande de congé s'enregistre, le solde se met à jour, le chef est notifié ; ou : l'absence est marquée, l'employé joint son justificatif, le chef le valide).

- Un point de 15 minutes deux fois par semaine : ce que j'ai fait, ce que je vais faire, ce qui me bloque.
- Une intégration chaque semaine : on fusionne tout et on teste les parcours complets :
  - **Congé** : l'employé demande, le chef donne son avis, la RH approuve.
  - **Absence** : le chef la marque, l'employé justifie, le chef valide, la retenue éventuelle apparaît sur la fiche.
  - **Paie** : le chef valide les heures supplémentaires, la RH les contrôle, la fiche se génère puis est validée et envoyée.
- Les deux dernières semaines sont réservées à la sécurité, au responsive (360, 768 et 1280 px), au rapport, au manuel et à la répétition de la soutenance, pas à de nouvelles fonctions.

> **Ordre de développement conseillé :** authentification et droits (A), puis employés / départements / contrats (B), puis congés et absences (C), puis paie (D), puis interactions JavaScript et tableau de bord, puis e-mails et exports, puis documentation. Pendant que A termine l'authentification, B, C et D travaillent sur leur schéma et leurs écrans avec les comptes de démo.

7. Pièges à éviter

- Modifier des tables ou des champs sans prévenir le groupe.
- Commencer les extras (avances, justificatifs d'absence, tableau de bord, coffre-fort, Google) avant que le noyau soit fini : le barème note d'abord les 12 fonctionnalités.
- Coder un écran qui n'existe pas dans les maquettes sans l'avoir dessiné et validé avant.
- Repousser l'intégration « à la fin ».
- Laisser une personne devenir le goulot d'étranglement : A doit finir l'authentification en premier, sinon tout le monde est bloqué.

8. Prochaines actions

- Confirmer avec le professeur que le groupe peut être de 4 (le sujet parle de binôme ou trinôme).
- Choisir qui prend A, B, C et D.
- Créer le dépôt Git et la structure MVC.
- Valider le modèle de 25 tables (garder ou retirer les compléments et extras), puis geler le script SQL.
- Écrire le README des conventions.
- Convenir des signatures des fonctions partagées et du format JSON.
- Terminer les maquettes (écrans Employé et Chef corrigés, puis RH et DG).
- Créer le tableau de tâches et fixer les jours des points d'équipe.